Università degli Studi Roma Tre

Dipartimento di Ingegneria Civile, Informatica e delle Tecnologie Aeronautiche

Corso di Deep Learning

Docente: Prof. Fabio Gasparetti

Project 1.2

Structured Information Extraction

Report del progetto

Autori

Alfredo Carta
Juan Miguel Pinos Seco

Repository del progetto:

`https://github.com/JuanMiPS/Structured-Information-Extraction/tree/main`

Anno Accademico 2025–2026

Sommario

Questo progetto affronta il task di *structured information extraction*: data una descrizione testuale in linguaggio naturale, il modello deve produrre un output in formato strutturato. L'obiettivo principale è garantire (i) **validità sintattica dell'output JSON prodotto**, (ii) **aderenza rigorosa allo schema** richiesto e (iii) **riduzione delle allucinazioni** nei campi estratti.

L'esperimento utilizza il dataset pubblico **RecipeNLG** e due Small Language Models: **Qwen2.5-3B-Instruct** e **Mistral-7B-Instruct-v0.3**. Dopo una valutazione iniziale in modalità **zero-shot**, i modelli vengono sottoposti a **fine-tuning leggero tramite QLoRA su modelli caricati in quantizzazione 4-bit**.

Le prestazioni vengono valutate tramite metriche automatiche che misurano sia la correttezza sintattica sia la qualità semantica delle informazioni estratte. I risultati mostrano un miglioramento delle prestazioni dopo il fine-tuning, dimostrando l'efficacia di tecniche di adattamento efficienti anche su modelli di dimensioni relativamente ridotte.

Indice

1 Introduzione **2**
2 Requisiti del progetto e corrispondenza con l'implementazione **2**
3 Workflow sperimentale **3**
4 Pipeline del sistema **3**
5 Task e schema JSON **4**
5.1 Definizione del problema 4
5.2 Schema JSON target 4
6 Dataset **4**
7 Preprocessing dei dati **5**
8 Modelli utilizzati **5**
8.1 Architettura dei modelli 5
8.2 Mistral-7B-Instruct 6
8.3 Qwen2.5-3B-Instruct 6
9 Fine-tuning con QLoRA **6**
9.1 QLoRA: fine-tuning efficiente 6
10 Implementazione tecnica del prompting e della generazione **7**
11 Metriche **8**
11.1 Interpretazione delle metriche 8
12 Risultati **8**
13 Discussione **9**
14 Limitazioni **9**
15 Conclusioni **10**

1 Introduzione

Negli ultimi anni i Large Language Models hanno mostrato risultati molto rilevanti in numerosi compiti di Natural Language Processing, tra cui classificazione, question answering, riassunto automatico e generazione di testo. Un aspetto particolarmente interessante è la loro capacità di trasformare testo non strutturato in rappresentazioni strutturate, come tabelle o oggetti JSON, tramite istruzioni formulate nel prompt.

In questo progetto il problema affrontato consiste nell'estrarre informazioni strutturate da descrizioni testuali di ricette culinarie. L'obiettivo non è soltanto generare un testo plausibile, ma costruire un output formalmente corretto e semanticamente utile, con campi ben definiti e compatibili con un processo automatico di parsing e valutazione.

Più precisamente, a partire da una ricetta espressa in linguaggio naturale, il sistema deve restituire un JSON contenente una lista di ingredienti, il tempo di cottura in minuti e la temperatura di preparazione, quando presente. Questo tipo di task è particolarmente interessante perché combina due aspetti tipici delle applicazioni reali dei modelli generativi: da un lato la comprensione del contenuto del testo, dall'altro il rispetto di vincoli sintattici rigidi.

Dal punto di vista sperimentale, il progetto confronta due Small Language Models, Qwen2.5-3B-Instruct e Mistral-7B-Instruct-v0.3, in due condizioni: prima del fine-tuning e dopo una fase di adattamento efficiente tramite QLoRA. In questo modo è possibile analizzare quanto un fine-tuning leggero riesca a specializzare modelli già preaddestrati in un task di structured information extraction.

2 Requisiti del progetto e corrispondenza con l'implementazione

Il progetto richiede di:

- tradurre testo naturale in formato strutturato (JSON consigliato);
- curare: **validità sintattica**, **aderenza allo schema**, **riduzione delle allucinazioni**;
- seguire la metodologia: **dataset** → **benchmark** → **2/3 SLM** → risultati **PRE** → **fine-tuning** → risultati **POST**.

L'intera implementazione del progetto è disponibile nel repository:

`https://github.com/JuanMiPS/Structured-Information-Extraction/tree/main`

Nel repository sono presenti:

- il notebook con l'implementazione completa della pipeline sperimentale;
- il codice per la valutazione e la generazione delle metriche;
- i file di output generati dagli esperimenti.

Il progetto soddisfa i requisiti richiesti costruendo una pipeline completa che parte dal dataset, definisce un task di generazione strutturata, confronta due modelli di dimensioni differenti, esegue una valutazione iniziale zero-shot, applica un fine-tuning efficiente e infine misura l'effetto dell'adattamento tramite metriche dedicate.

3 Workflow sperimentale

La pipeline sperimentale seguita nel progetto è la seguente:

- Selezione del dataset (**RecipeNLG**)
- Definizione del prompt e dello schema JSON target
- Preprocessing dei dati e costruzione di etichette strutturate
- Valutazione iniziale dei modelli in modalità zero-shot (PRE evaluation)
- Fine-tuning leggero dei modelli tramite QLoRA
- Valutazione finale dopo il fine-tuning (POST evaluation)
- Confronto quantitativo dei risultati

La pipeline prevede quindi una valutazione iniziale dei modelli senza addestramento specifico (zero-shot), seguita da un fine-tuning leggero tramite QLoRA e da una successiva valutazione comparativa. Questo approccio consente di distinguere chiaramente ciò che il modello sa già fare grazie al pretraining generale da ciò che apprende in modo specifico durante la fase di adattamento sul task.

4 Pipeline del sistema

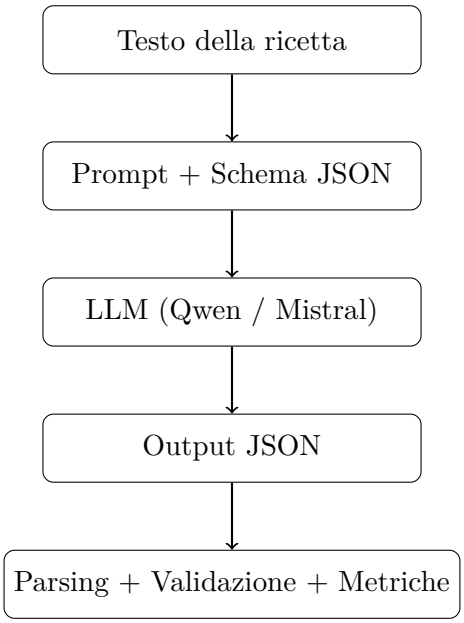

Figura 1: Pipeline del sistema di estrazione strutturata.

La figura sintetizza i passaggi essenziali del sistema. A partire da una ricetta in linguaggio naturale, viene costruito un prompt che include istruzioni esplicite e uno schema JSON desiderato. Il modello genera quindi un output che viene successivamente sottoposto a parsing e validazione. Infine, l'output strutturato viene confrontato con la label target per ottenere le metriche di valutazione.

5 Task e schema JSON

5.1 Definizione del problema

Dato un input testuale che descrive una ricetta, il modello deve produrre un JSON con i seguenti campi:

- `ingredients`: lista di ingredienti;
- `cooking_time`: tempo di cottura in minuti oppure `null`;
- `temperature`: oggetto contenente valore e unità di temperatura oppure `null`.

Il problema può essere interpretato come una forma di generazione strutturata vincolata: il modello deve comprendere il contenuto semantico del testo e, allo stesso tempo, rispettare una struttura di output molto rigida. Questa doppia richiesta rende il task più complesso di una semplice generazione testuale libera.

5.2 Schema JSON target

Listing 1: Schema JSON atteso

```
{
  "ingredients": ["string", "..."],
  "cooking_time": 123,
  "temperature": {"value": 180, "unit": "C"}
}
```

L'uso di uno schema JSON fisso consente di valutare separatamente due aspetti: la correttezza formale dell'output e l'accuratezza delle informazioni estratte. In particolare, è possibile verificare se il modello produce un JSON valido e se inserisce esattamente i campi richiesti senza aggiungere chiavi non previste.

6 Dataset

È stato utilizzato il dataset pubblico **RecipeNLG**, che contiene descrizioni testuali di ricette.

RecipeNLG è un dataset pubblico progettato per task di generazione e di estrazione di informazioni da ricette. Il dataset include descrizioni testuali di ingredienti, istruzioni di preparazione e metadati culinari, rendendolo adatto a esperimenti di structured information extraction.

Per rendere l'esperimento eseguibile su hardware limitato (Google Colab con GPU Tesla T4), è stato utilizzato un **subset del dataset**:

- Train: 300 esempi
- Test: 40 esempi

Lo split è deterministico tramite seed fissato per garantire la riproducibilità degli esperimenti.

L'utilizzo di un subset relativamente piccolo rappresenta un compromesso tra costo computazionale e rapidità sperimentale. Sebbene un numero maggiore di esempi avrebbe probabilmente migliorato la robustezza del fine-tuning, la configurazione scelta consente di eseguire l'intera pipeline in ambiente Colab mantenendo tempi e risorse compatibili con un progetto accademico.

7 Preprocessing dei dati

Prima di utilizzare il dataset per l'addestramento e la valutazione, è stato necessario applicare alcune operazioni di preprocessing. In particolare, il testo delle ricette è stato ripulito da caratteri ridondanti, spazi multipli e formattazioni incoerenti. Gli ingredienti sono stati normalizzati e trasformati in una rappresentazione più uniforme, così da ridurre il rumore presente nei dati originari.

Una parte importante del preprocessing consiste nella costruzione delle etichette target. Poiché il dataset non fornisce direttamente il JSON richiesto dal task, le etichette vengono derivate automaticamente a partire dai campi della ricetta. Gli ingredienti vengono ricavati dalle liste presenti nel dataset, mentre il tempo di cottura e la temperatura vengono estratti tramite semplici regole basate su espressioni regolari. Questo approccio produce delle *silver labels*, ossia annotazioni generate automaticamente che non hanno la precisione di un'annotazione manuale ma sono sufficienti per costruire un benchmark coerente.

L'uso di silver labels è una scelta metodologica importante: consente di eseguire il fine-tuning senza la necessità di annotazioni umane, ma introduce anche rumore. Questo aspetto viene ripreso nella sezione delle limitazioni.

8 Modelli utilizzati

Sono stati utilizzati due Small Language Models:

- **Qwen2.5-3B-Instruct**
- **Mistral-7B-Instruct-v0.3**

Entrambi i modelli sono stati caricati in quantizzazione **4-bit** tramite `bitsandbytes` per ridurre l'uso di memoria GPU.

8.1 Architettura dei modelli

Entrambi i modelli utilizzati in questo progetto appartengono alla famiglia dei **Transformer-based Large Language Models**. I transformer sono architetture neurali introdotte da Vaswani et al. (2017) nel lavoro *Attention is All You Need* e rappresentano oggi lo standard per la maggior parte dei modelli di NLP.

L'idea principale dei transformer è l'utilizzo del meccanismo di **self-attention**, che permette al modello di analizzare le relazioni tra tutte le parole di una frase simultaneamente. Questo consente al modello di catturare dipendenze a lungo raggio all'interno del testo, migliorando significativamente la capacità di comprensione del linguaggio naturale.

Un transformer è composto principalmente da tre componenti fondamentali:

- **Embedding layer**, che trasforma i token in vettori numerici;
- **Blocchi di self-attention**, che modellano le relazioni tra i token della sequenza;
- **Feed-forward layers**, che applicano trasformazioni non lineari alle rappresentazioni interne del modello.

Queste architetture vengono addestrate su grandi quantità di dati testuali per apprendere rappresentazioni generali del linguaggio. Successivamente possono essere adattate a task specifici tramite tecniche di fine-tuning o prompting.

8.2 Mistral-7B-Instruct

Mistral-7B è un modello di linguaggio open-source con circa 7 miliardi di parametri sviluppato da Mistral AI. Il modello è basato su un'architettura transformer ottimizzata che introduce diverse migliorie rispetto ai modelli precedenti.

Una delle caratteristiche principali di Mistral è l'utilizzo di tecniche di ottimizzazione dell'attenzione, come lo **sliding window attention**, che permette di ridurre il costo computazionale mantenendo una buona capacità di modellare il contesto. In termini pratici, questo aiuta il modello a gestire sequenze relativamente lunghe in modo efficiente.

La versione **Mistral-7B-Instruct** è stata ulteriormente adattata tramite instruction tuning, un processo di addestramento che insegna al modello a seguire istruzioni esplicite fornite nel prompt. Questo lo rende particolarmente adatto a task in cui è necessario produrre output controllati e coerenti con richieste ben precise.

Nel contesto del progetto, Mistral rappresenta il modello più grande tra i due testati e, come mostrano i risultati, parte da una qualità iniziale più alta in fase PRE, soprattutto per quanto riguarda l'estrazione semantica degli ingredienti.

8.3 Qwen2.5-3B-Instruct

Qwen2.5-3B è un modello di linguaggio sviluppato dal gruppo Alibaba Cloud. Il modello appartiene alla famiglia Qwen ed è progettato per offrire buone prestazioni anche con dimensioni relativamente ridotte.

Con circa 3 miliardi di parametri, Qwen2.5-3B è significativamente più piccolo rispetto ad altri LLM moderni, ma mantiene una buona capacità di comprensione e generazione del linguaggio naturale. Questo lo rende particolarmente interessante in scenari in cui la disponibilità di risorse hardware è limitata.

La versione **Instruct** del modello è stata addestrata con tecniche di instruction tuning, che migliorano la capacità del modello di seguire prompt strutturati e di generare output aderenti alle richieste. In contesti accademici o applicativi a costo ridotto, modelli di questa fascia risultano particolarmente utili perché possono essere adattati in modo rapido pur mantenendo prestazioni competitive.

Nel progetto, Qwen2.5-3B mostra un miglioramento molto evidente dopo il fine-tuning, dimostrando che anche modelli più piccoli possono essere specializzati con successo in compiti strutturati.

9 Fine-tuning con QLoRA

Il fine-tuning è stato eseguito tramite la tecnica **QLoRA**, che consente di addestrare modelli di grandi dimensioni aggiornando solo piccoli adapter LoRA.

9.1 QLoRA: fine-tuning efficiente

Il fine-tuning dei Large Language Models richiede normalmente un grande consumo di memoria e di risorse computazionali. Per questo motivo in questo progetto è stata utilizzata la tecnica **QLoRA** (*Quantized Low-Rank Adaptation*).

QLoRA combina due tecniche principali.

Quantizzazione. La quantizzazione consiste nel rappresentare i pesi del modello con un numero ridotto di bit. Nel nostro caso il modello viene caricato in precisione **4-bit**. Questo riduce significativamente il consumo di memoria GPU e permette di eseguire il modello anche su hardware limitato come una GPU Tesla T4. La quantizzazione non elimina il comportamento del modello, ma lo rende più compatto ed efficiente dal punto di vista computazionale.

LoRA (Low-Rank Adaptation). Invece di aggiornare tutti i parametri del modello, LoRA introduce delle piccole matrici di adattamento che vengono inserite all'interno di alcune componenti dell'architettura transformer. Durante il fine-tuning, i pesi originali del modello rimangono congelati, mentre vengono aggiornati solamente questi nuovi parametri aggiuntivi.

Questo approccio riduce drasticamente il numero di parametri da addestrare e permette di adattare il modello a nuovi task con un costo computazionale molto inferiore rispetto al fine-tuning tradizionale. In pratica, il modello conserva la conoscenza generale appresa nel pretraining e aggiunge una specializzazione locale per il task specifico.

Configurazione LoRA:

- rank $r = 8$
- alpha = 16
- dropout = 0.05

Questi valori controllano la capacità degli adapter LoRA e rappresentano un compromesso tra efficienza computazionale e capacità di adattamento del modello.

L'addestramento è stato eseguito con **un numero limitato di step** per contenere il tempo di addestramento e mantenere l'esperimento compatibile con risorse computazionali limitate. Pur non essendo un setup pensato per ottenere il massimo assoluto delle prestazioni, questa configurazione è adeguata per mostrare l'efficacia del fine-tuning efficiente in un contesto realistico.

10 Implementazione tecnica del prompting e della generazione

Per guidare i modelli verso un output strutturato, il sistema utilizza un prompt esplicito che descrive chiaramente il formato richiesto. Il prompt specifica che il modello deve produrre un singolo oggetto JSON, senza testo aggiuntivo, e che devono essere presenti esattamente i campi previsti dallo schema.

Questo accorgimento è fondamentale perché i modelli di linguaggio, se lasciati liberi, tendono a generare spiegazioni, testo ridondante o formattazioni miste che renderebbero più difficile il parsing automatico. L'uso di un prompt rigido riduce quindi la probabilità di errore sintattico.

Anche in presenza di istruzioni molto precise, tuttavia, il modello può occasionalmente produrre output non perfettamente conformi. Per questa ragione, il codice include una fase di parsing che cerca di identificare il blocco JSON all'interno dell'output generato, seguita da una fase di validazione dello schema. Solo dopo questo passaggio le predizioni vengono utilizzate per il calcolo delle metriche.

La generazione nella fase di valutazione viene eseguita in modo deterministico, con parametri come `temperature = 0` e `do_sample = False`. Questo serve a rendere le misure stabili e riproducibili, riducendo la componente casuale della generazione.

11 Metriche

Le metriche utilizzate sono:

- `valid_json_rate`
- `schema_ok_rate`
- `ingredient_f1`
- `cooking_time_accuracy`
- `temperature_accuracy`

Queste metriche permettono di valutare sia la correttezza sintattica dell'output JSON sia la correttezza semantica delle informazioni estratte.

11.1 Interpretazione delle metriche

La metrica `valid_json_rate` misura la percentuale di output che possono essere interpretati correttamente come JSON validi. Essa cattura quindi la correttezza sintattica dell'output prodotto dal modello.

La metrica `schema_ok_rate` è più restrittiva: verifica che il JSON valido contenga esattamente i campi richiesti dallo schema e non includa chiavi extra o mancanti. In questo modo si misura l'aderenza formale al task.

La metrica `ingredient_f1` valuta la qualità dell'estrazione della lista degli ingredienti, combinando precisione e richiamo. È una metrica particolarmente utile perché gli ingredienti sono una lista e non un singolo valore.

Le metriche `cooking_time_accuracy` e `temperature_accuracy` misurano invece l'accuratezza esatta dei valori estratti per tempo di cottura e temperatura. Si tratta quindi di una valutazione più rigida, ma appropriata per campi numerici o quasi numerici.

12 Risultati

Modello	**valid JSON**	**schema OK**	**ingredient F1**	**time acc**	**temp acc**
Qwen 3B (PRE)	1.000	1.000	0.7159	0.600	0.575
Qwen 3B (POST)	1.000	1.000	0.9583	0.650	0.925
Mistral 7B (PRE)	1.000	1.000	0.9079	0.500	0.675
Mistral 7B (POST)	0.975	0.975	0.9750	0.875	0.925

Tabella 1: Confronto delle prestazioni PRE vs POST fine-tuning sul set di valutazione (40 esempi).

I risultati mostrano un miglioramento significativo delle metriche semantiche dopo il fine-tuning per entrambi i modelli. In particolare, si osserva un aumento marcato dell'*ingredient F1*, che misura la correttezza dell'estrazione degli ingredienti, indicando che i modelli riescono a identificare con maggiore precisione le informazioni rilevanti nel testo. Anche le metriche relative agli attributi strutturati (*time accuracy* e *temperature accuracy*) migliorano sensibilmente dopo l'addestramento specifico sul task.

Nel caso di **Qwen 3B**, l'incremento della metrica *ingredient F1* da 0.7159 a 0.9583 è particolarmente rilevante. Questo risultato suggerisce che il modello, pur partendo da una base più

modesta rispetto a Mistral, beneficia in maniera marcata dell'adattamento tramite QLoRA. Anche la *temperature accuracy* migliora in modo molto netto, passando da 0.575 a 0.925. Ciò indica che il fine-tuning permette al modello di apprendere meglio i pattern con cui le temperature vengono espresse all'interno delle ricette.

Per quanto riguarda **Mistral 7B**, il modello mostra già in fase PRE prestazioni iniziali elevate, specialmente nell'estrazione degli ingredienti. Dopo il fine-tuning, la metrica *ingredient F1* raggiunge 0.9750, mentre la *time accuracy* cresce in modo molto consistente da 0.500 a 0.875. Questo suggerisce che il modello, grazie alla sua maggiore capacità rappresentativa, riesce a sfruttare il fine-tuning per migliorare in modo significativo l'estrazione delle informazioni numeriche o semi-strutturate.

Per quanto riguarda la validità sintattica, entrambi i modelli mantengono valori molto elevati nelle metriche *valid JSON* e *schema OK*, indicando che sono generalmente in grado di rispettare il formato strutturato richiesto. Si osserva una lieve diminuzione per Mistral 7B nella configurazione POST, che potrebbe essere dovuta a una maggiore variabilità generativa dopo l'addestramento. Tuttavia, il calo è molto contenuto e non compromette il miglioramento complessivo delle metriche semantiche.

Nel complesso, i risultati confermano che il fine-tuning consente di migliorare significativamente la qualità dell'estrazione strutturata mantenendo al contempo una buona affidabilità nella generazione del formato JSON. L'esperimento evidenzia inoltre che sia i modelli più piccoli sia quelli di dimensioni maggiori possono trarre beneficio da tecniche di adattamento efficienti, anche quando il dataset disponibile è relativamente limitato.

13 Discussione

I risultati ottenuti permettono alcune considerazioni interessanti. In primo luogo, i valori molto elevati di *valid JSON* e *schema OK* già nella fase PRE indicano che i modelli istruiti moderni possiedono una capacità iniziale piuttosto buona di seguire istruzioni formali e di produrre output strutturati. Questo è coerente con la natura *instruction-tuned* dei modelli impiegati.

In secondo luogo, il miglioramento osservato nelle metriche semantiche dimostra che il fine-tuning non agisce tanto sul rispetto del formato, che era già elevato, quanto sulla capacità di collegare in modo più accurato le espressioni linguistiche del testo ai valori da inserire nel JSON. In altre parole, l'adattamento non serve principalmente a far "parlare meglio" il modello, ma a farlo aderire meglio al task specifico.

Un ulteriore aspetto interessante riguarda la differenza tra i due modelli. Mistral 7B parte da una base più forte, ma Qwen 3B mostra un guadagno relativo molto marcato. Questo risultato è particolarmente rilevante dal punto di vista applicativo: suggerisce che, in scenari con risorse limitate, anche un modello più piccolo può essere specializzato con successo fino a raggiungere prestazioni molto competitive.

14 Limitazioni

Il dataset utilizzato per il fine-tuning è relativamente piccolo (300 esempi di training), e questo limita la capacità dei modelli di apprendere pattern più complessi.

Inoltre le etichette utilizzate derivano da un processo di parsing automatico delle ricette e non da annotazioni manuali, il che può introdurre rumore nei dati. La presenza di silver labels rende il benchmark utile e veloce da costruire, ma non garantisce la stessa affidabilità di un dataset annotato manualmente.

Un'ulteriore limitazione è rappresentata dal numero ridotto di esempi nel set di valutazione. Con soli 40 esempi, anche una piccola variazione nel numero di predizioni corrette può avere un impatto

visibile sulle metriche finali. Pertanto i risultati vanno interpretati come indicativi dell'efficacia della pipeline, più che come una misura definitiva delle prestazioni assolute dei modelli.

Esperimenti futuri potrebbero utilizzare dataset più ampi, annotazioni manuali o tecniche di decoding ancora più vincolate, così da migliorare ulteriormente sia la qualità dell'estrazione semantica sia la robustezza formale dell'output JSON.

15 Conclusioni

Il progetto dimostra come tecniche di fine-tuning efficienti come **QLoRA** possano migliorare significativamente le prestazioni dei Small Language Models su task di *structured information extraction*.

Nonostante l'utilizzo di un dataset di dimensioni ridotte, entrambi i modelli mostrano miglioramenti evidenti nelle metriche di estrazione semantica dopo il fine-tuning. In particolare, l'aumento dell'*ingredient F1*, della *time accuracy* e della *temperature accuracy* conferma che il task può essere appreso efficacemente anche con una quantità contenuta di dati e con risorse hardware limitate.

Dal punto di vista metodologico, il progetto mostra anche l'importanza di considerare separatamente la correttezza sintattica dell'output e la correttezza semantica delle informazioni estratte. Un sistema di structured generation efficace deve infatti soddisfare entrambe le condizioni: produrre un formato formalmente valido e contenere valori realmente coerenti con il testo di partenza.

Questi risultati evidenziano come tecniche di adattamento efficienti possano essere sufficienti per specializzare modelli generativi su compiti strutturati specifici, anche utilizzando risorse computazionali limitate. In prospettiva, l'estensione del dataset, l'impiego di annotazioni gold e l'introduzione di metodi di generazione vincolata potrebbero costituire passi successivi naturali per migliorare ulteriormente il sistema.